Prior-Auth Hackathon: Architecture & Product Spec

Oct 13, 2026 · @Ullhas Y Kumar

Overview: One product, three merged concepts

The site is a single Prior-Authorization workspace with three layers that correspond to Concepts A, C and E, all sharing one extraction engine so there is no duplicated logic:

1. **Intake & Extraction (Concept A)** — a staff member uploads a patient's clinical documentation (notes, discharge summary, lab reports) and selects the payer/procedure. The engine extracts every field the target PA form needs, cites the exact source sentence for each one, and flags what's missing before submission.
2. **Triage Dashboard (Concept C)** — the doctor/nurse-facing home screen. Every case in progress shows up as a card or row: Ready to Submit, Needs Info, Submitted, Denied. Bulk view across all active patients, not one case at a time.
3. **Denial & Appeal Assistant (Concept E)** — when a payer denial comes back, staff upload the denial letter next to the original chart; the same extraction engine re-reads the chart against the stated denial reason, finds the specific missing evidence, and drafts the appeal with citations.

All three share one data model (a *Case*) and one AI pipeline (extract → cite → gap-check). The dashboard is the hub; intake and appeal are actions taken on a Case from that hub.

Doctor/Nurse Dashboard — Sections

One screen, one Case, six sections top to bottom. Each section is collapsible so a user working a known case can skip straight to what's unresolved.

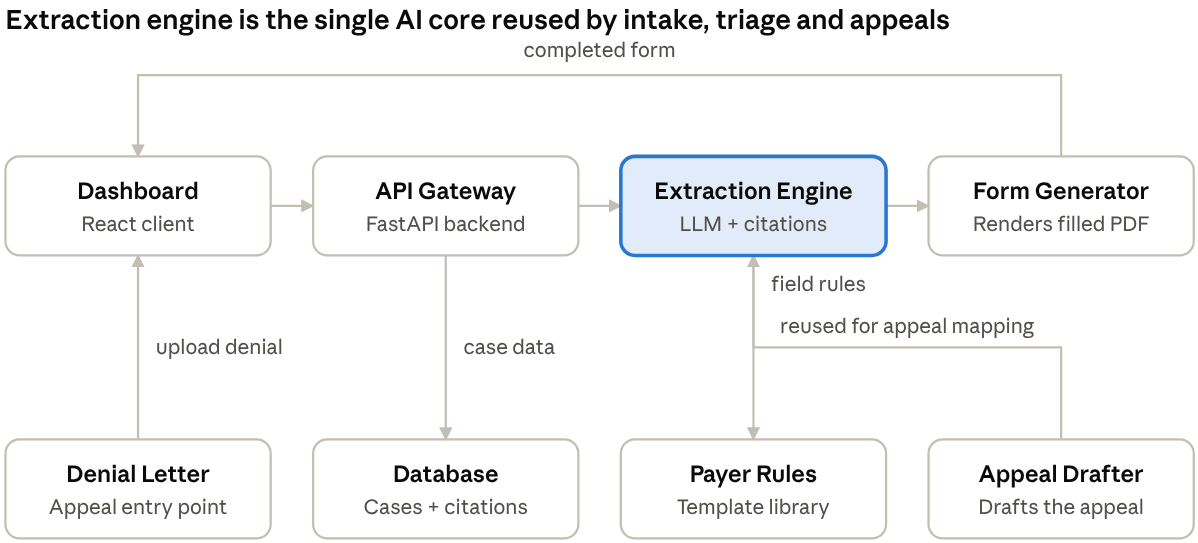
#	Section	What it shows	Who acts on it
1	**Case Header**	Patient name, DOB, MRN, procedure requested, target payer, current status chip (Ready / Needs Info / Submitted / Denied / Appealing)	Everyone — orientation only

#	Section	What it shows	Who acts on it
2	**Completeness Checklist**	Every required field for the selected payer form, each row: field name, auto-filled value, confidence (High/Review/Missing), and a **"View Source"** link	Nurse/staff — confirms or edits each field
3	**Source Viewer (side panel)**	The original clinical document with the cited sentence highlighted, opened by clicking "View Source" on any field	Nurse/doctor — verifies AI didn't hallucinate
4	**Missing Information**	Only the fields the AI could not find, each with a one-line reason ("No mention of conservative treatment duration") and a button to request it from the ordering physician	Nurse — triggers follow-up; Doctor — responds
5	**Diagnosis & Procedure Coding**	AI-suggested ICD-10 / ICD-10-PCS codes with confidence and source citation, editable before submit (this is the highest-risk section — never auto-submit without human confirmation here)	Doctor — final clinical sign-off
6	**Submission & History**	Final generated PA form (preview + download), submit button, and a timeline of this case's events (extracted → flagged → doctor reviewed → submitted → denied → appeal drafted)	Nurse — submits; everyone — reads history

Above the single-case view sits the **Triage Queue** (Concept C), the landing page after login: a table of all active cases with columns Patient, Procedure, Payer, Status, Days Open, and a sort/filter by status so staff can immediately see which cases are blocked on missing info versus ready to go. Clicking a row opens that case's six-section view above.

When a case's status is **Denied**, section 6 is replaced by the **Appeal Builder** (Concept E): upload the denial letter, the AI maps the stated denial reason to specific weak/missing evidence in the chart (reusing the same extraction engine), and drafts an appeal letter with citations for review before sending.

System Architecture



system architecture · 8 components, one shared AI core

The Extraction Engine is the only AI component in the system — both the first-pass submission (Concept A) and the denial appeal (Concept E) reuse it, and the Triage Dashboard (Concept C) is simply the queue of Cases sitting on top of this pipeline. The Database also feeds the Form Generator directly (not drawn, to keep the picture readable), since every filled field has to be re-read when the final PDF is rendered.

AI Extraction & Source-Citation Pipeline

This is the part that has to be trustworthy, not just fast. Every field the AI fills carries a pointer back to the exact source text — nothing is accepted on the AI's word alone.

Step-by-step for one case:

1. **Ingest** — clinical document(s) uploaded (PDF/text) are parsed into plain text, with each sentence assigned a stable `source_id` (document id + character offset range).

2. **Load the target schema** — the selected payer + procedure combination pulls its required-field list from the Payer Template Library (a JSON schema: field name, data type, required/optional, valid values).
3. **Structured extraction call** — the LLM is prompted with the full note text + the schema, and required to return JSON matching this per-field shape:

```
{
  "field": "primary_diagnosis_icd10",
  "value": "M54.5",
  "value_label": "Low back pain",
  "source_id": "doc_2:para_14:char_210-265",
  "source_text": "Patient presents with chronic low back pain...",
  "confidence": "high"
}
```

4. **Validation pass (code, not AI)** — a deterministic checker confirms `source_text` is an exact substring found at `source_id` in the original document. If the LLM's cited source doesn't actually contain that text, the field is force-downgraded to `confidence: "review"` regardless of what the LLM claimed — this is the single most important safeguard against hallucination.
5. **Gap detection** — any required field from the schema with no extraction result is marked `missing`, with a one-line system-generated reason ("no mention of failed conservative treatment found").
6. **Render to dashboard** — the Completeness Checklist (dashboard section 2) shows field, value, confidence badge, and a "View Source" link that opens the Source Viewer (section 3) scrolled and highlighted to that exact `source_id`.

Reused, not rebuilt, for the Appeal Builder (Concept E): the denial letter is parsed the same way, the stated denial reason is matched against the schema to identify which field(s) likely triggered it, and step 3's extraction is re-run specifically against those fields with a prompt asking "does the chart contain stronger evidence for X than what was originally submitted?" — same validation pass, same citation format, same UI pattern.

Confidence levels shown to staff: `high` (source text found, matches field type) · `review` (AI found something but validation couldn't confirm the exact quote, or the value is a coded field like ICD-10) · `missing` (schema field required, nothing found). Only `high` fields are ever pre-filled into the final form without a visible flag; `review` and `missing` always block one-click submit.

Data Model

Six tables cover all three concepts — no concept needs its own schema.

Table	Key fields	Purpose
`cases`	id, patient_name, dob, mrn, procedure, payer_id, status (ready/needs_info/submitted/denied/appealing), created_at	One row per PA request; the thing the Triage Dashboard lists
`documents`	id, case_id, type (clinical_note/discharge_summary/denial_letter), raw_text, uploaded_at	Every uploaded source file, parsed to text
`payer_templates`	id, payer_name, procedure_type, schema_json (required fields + types)	The field requirements per payer/procedure — Concept A's "what counts as complete"
`extracted_fields`	id, case_id, field_name, value, confidence (high/review/missing), source_doc_id, source_offset_start, source_offset_end, source_text	One row per form field; this table is what Section 2 and the Source Viewer render from
`case_events`	id, case_id, event_type, note, actor, timestamp	Audit timeline — Section 6's history, and what the Triage Dashboard's "Days Open" is computed from
`appeals`	id, case_id, denial_reason_text, denial_doc_id, matched_fields (array of extracted_field ids), draft_text	One row per appeal attempt — Concept E, reusing `extracted_fields` rather than duplicating extraction

The foreign-key shape is deliberately flat: `documents`, `extracted_fields`, `case_events`, and `appeals` all key off `case_id` directly. That's what lets the dashboard assemble a full case view with simple queries instead of multi-level joins — important when you only have hours to build and debug it.

API Endpoints

Method	Path	Used by	Does
GET	`/cases`	Triage Dashboard	List all cases with status, patient, payer, days-open — the queue table
POST	`/cases`	Intake (Concept A)	Create a case: patient info + payer/procedure selection
POST	`/cases/{id}/documents`	Intake	Upload a clinical document; triggers ingest + parse
POST	`/cases/{id}/extract`	Intake	Runs the extraction pipeline against all uploaded documents for this case
GET	`/cases/{id}`	Case view (Dashboard sections 1–6)	Full case: fields, confidence, missing list, events
GET	`/cases/{id}/fields/{field_id}/source`	Source Viewer (section 3)	Returns the source document + exact highlighted offset range
PATCH	`/cases/{id}/fields/{field_id}`	Doctor sign-off (section 5)	Staff/doctor edits or confirms a field value before submit
POST	`/cases/{id}/request-info`	Missing Info (section 4)	Logs a follow-up request to the ordering physician for a missing field

Method	Path	Used by	Does
POST	`/cases/{id}/submit`	Submission (section 6)	Generates the final PDF, marks case `submitted`, logs event
GET	`/cases/{id}/form.pdf`	Submission	Downloads the generated PA form
POST	`/cases/{id}/denial`	Appeal Builder (Concept E)	Upload denial letter; marks case `denied`, triggers denial-reason extraction
POST	`/cases/{id}/appeal/draft`	Appeal Builder	Runs the reused extraction pipeline against the denial reason, returns draft appeal text with citations
POST	`/cases/{id}/appeal/send`	Appeal Builder	Marks case `appealing`, logs event, downloads final appeal letter

All extraction-backed endpoints (`/extract`, `/appeal/draft`) return the same per-field JSON shape shown in the pipeline section above, so the frontend renders the Completeness Checklist and the Appeal evidence list with one shared component.

Tech Stack

Layer	Choice	Why, under an 18-hour clock
Frontend	React + Vite, Tailwind	Fastest path to a dashboard with cards/tables/panels; no build-config time lost

Layer	Choice	Why, under an 18-hour clock
Backend	FastAPI (Python)	Same language as the extraction/validation code — no context-switch between API and AI logic; auto-generates OpenAPI docs for free
Database	SQLite for the build, Postgres-compatible schema	SQLite needs zero setup time; schema is written so swapping to Postgres later is a connection-string change, not a rewrite
LLM API	Claude API (structured JSON output)	Reliable structured-output mode; the validation pass in the pipeline section is what actually guards against hallucination, not model choice
PDF generation	WeasyPrint or a headless HTML-to-PDF step	Renders the filled form as styled HTML, then to PDF — avoids hand-building a PDF layout engine
Payer templates	Hardcoded JSON files in-repo (3–5 templates)	No real payer API exists to integrate; this is the explicitly mocked piece — label it as such on the pitch slide, don't hide it
Hosting (demo)	Local / single cloud VM (Render, Railway, or a laptop + ngrok)	No need for scalable infra for a judged demo; deploy only if the submission requires a live link
Auth	Skip, or a single hardcoded demo login	Multi-user auth adds hours of work the judging criteria almost certainly don't reward in an 18h window

The one component worth protecting time for is the **validation pass** (the code-based substring check in the pipeline section) — it's cheap to build and it's the single piece of engineering that answers "how do you know the AI isn't making this up," which is the hardest question a healthcare-AI judge will ask.

Build Sequencing

Build in this order — each stage produces something demoable, so you're never more than an hour from a working checkpoint:

1. **Schema + one payer template + one synthetic clinical note.** Nothing else matters until this exists.
2. **Extraction pipeline, called from a script (no UI yet).** Prove field + citation + confidence output works on real data before building anything else around it.
3. **Validation pass.** The substring-check safeguard — build this immediately after extraction works, not as a "polish" item later.
4. **Case view (dashboard sections 1–3): header, checklist, source viewer.** This alone is a demoable Concept A.
5. **Triage queue (dashboard landing page).** This alone, added on top of step 4, is Concept C.
6. **Missing-info + doctor sign-off + submit + PDF (sections 4–6).** Completes the end-to-end PA submission flow.
7. **Denial upload + appeal draft (Appeal Builder).** This alone, reusing the pipeline from step 2–3, is Concept E — build last since it depends on everything above existing first.
8. **Polish pass:** confidence badges, status chips, days-open sort — only after steps 1–7 all work end-to-end on at least one full case.

If time runs short, steps 1–6 alone are a complete, coherent, judge-ready demo (Concepts A + C). Step 7 (Concept E) is the first thing to cut, not the first thing to build — it's the differentiator, not the floor.